

Internship Interview — Questionnaire Submission

Role: AI/ML Engineer Intern | Backend Developer

Submitted To: Revathi Sankaran

Submitted By: Dhairya Rathore

Submission Date: 30 May 2026

Q1. Explain how Python is both compiled and interpreted. Why does this hybrid approach exist?

Python initially compiles your source code into bytecode (.pyc files), and then the Python interpreter (CPython) executes that bytecode line by line. This process occurs automatically in the background — the user does not need to initiate it manually.

This hybrid approach is practical because compiling to bytecode prevents Python from re-parsing the source code during each execution, saving time. Using a bytecode interpreter instead of compiling directly to machine code ensures the bytecode runs on any platform with Python installed, eliminating the need for platform-specific compilation.

Q2. What are lambda functions in Python? When would you use a lambda instead of a regular function?

A lambda is a short, anonymous function defined on a single line. It can accept any number of arguments but contains only one expression.

You'd use it when you need a quick throwaway function — like passing logic into `sorted()`, `map()`, or `filter()` — where writing a full `def` would just add noise.

```
nums = [3, 1, 2]
nums.sort(key=lambda x: -x) # sorts descending
```

If the logic spans more than one expression, a regular function is the cleaner choice.

Q3. How does JavaScript handle error handling? What are try, catch, and finally?

try runs the block of code you're testing. If any error is thrown, catch intercepts it and lets you handle it gracefully. finally always runs — regardless of whether an error occurred or not — making it useful for cleanup.

```
try {  
  JSON.parse(badInput);  
} catch (e) {  
  console.log('Parse failed:', e.message);  
} finally {  
  console.log('This always runs');  
}
```

Q4. Can you explain how JavaScript closures work? Give a real-world example.

A closure is when an inner function remembers variables from its outer function's scope, even after the outer function has finished executing.

Real-world example — a counter that keeps its own private state:

```
function makeCounter() {  
  let count = 0;  
  return () => ++count;  
}  
const counter = makeCounter();  
counter(); // 1  
counter(); // 2
```

count stays alive because the returned arrow function closed over it. This is also how things like React's useState and module-level private variables work under the hood.

Q5. Using a house analogy, describe what a function, module, class, and method are in Python.

Module = The entire house. It groups everything related together — just like a house holds all the rooms and utilities under one roof. In Python, a module is a single .py file.

Class = A specific room in the house — say, the kitchen. It has a defined purpose and holds everything relevant to that purpose: the appliances (attributes) and what you do in there (methods).

Method = An action performed inside that room — like turning on the stove or washing dishes. Methods are functions that belong to a class and work with its data.

Function = A standalone tool shed in the backyard. Useful on its own, not tied to any specific room. You call it from anywhere when you need it.

Q6. Explain the purpose of try, except, else, and finally in Python. Give examples.

```
try:
    result = 10 / x
except ZeroDivisionError:
    print('Cannot divide by zero')
else:
    print('Result:', result)
finally:
    print('Done')
```

try holds the code that might fail. except catches specific errors so the program doesn't crash. else keeps your success logic cleanly separate from error handling — it only runs when everything in try worked fine. finally is for teardown: closing files, releasing connections, etc.

Q7. Describe the purpose and functionality of groupby() in pandas. Provide examples.

groupby() splits a DataFrame into groups based on one or more column values, then lets you apply aggregate functions to each group. Think of it as SQL's GROUP BY.

```
# Average salary per department
df.groupby('department')['salary'].mean()
```

```
# Total sales per region
df.groupby('region')['sales'].sum()
```

```
# Count of orders per day
df.groupby('date')['order_id'].count()
```

It's useful whenever you need per-category summaries — aggregating, counting, or transforming data within defined groups.

Q8. Write a JavaScript function that reverses each word in a sentence but keeps the word order the same.

Input: 'peter piper picked a peck of pickled peppers'

Output: 'retep repip dekcip a kcep fo delkcip sreppep'

```
function reverseWords(sentence) {
  return sentence
    .split(' ')
    .map(word => word.split("").reverse().join(""))
    .join(' ');
}
```

Logic: split on spaces to get individual words, reverse the character array of each word, then join everything back. Word order never changes — only the characters inside each word are reversed.

Q9. What happens when you type a URL in your browser that hits a Django app? Walk through the request-response cycle.

1. Browser sends an HTTP request to the server.
2. Django's WSGI/ASGI layer receives the request.
3. The URL dispatcher checks `urls.py` and matches the path to a view.
4. The matched view function runs — it may query the database via models.
5. The view passes data to a template, which renders it into HTML.
6. Django sends back the HTTP response (HTML, JSON, etc.) to the browser.

Middleware runs at both ends — on the way in and on the way out — for things like auth, sessions, and CORS.

Q10. What are Django forms, and how can they be integrated with JavaScript for better UX?

Django forms handle server-side validation, field rendering, and CSRF protection. They reduce the boilerplate of manually validating POST data.

To improve UX, you submit them via `fetch()` instead of a full page reload. Django processes and validates the form, then returns JSON. JavaScript reads that response and updates only the relevant part of the page — no redirect, no flash.

```
fetch('/submit/', { method: 'POST', body: new FormData(form) })
  .then(r => r.json())
  .then(data => {
    if (data.success) showSuccessMessage();
    else showErrors(data.errors);
  });
```

Q11. What is the purpose of manage.py and settings.py in a Django project?

manage.py is Django's command-line entry point. You use it to start the dev server, run migrations, create superusers, run tests, and more.

settings.py is the project's central config file. Common things configured there:

INSTALLED_APPS: which apps are active

DATABASES: database engine, host, credentials

SECRET_KEY: used for cryptographic signing

ALLOWED_HOSTS: which domains can serve the app

STATIC_URL / MEDIA_URL: paths for static and uploaded files

MIDDLEWARE: the request/response processing pipeline

Q12. How would you implement fast, relevance-ranked search across blog post titles, tags, and content in Django?

PostgreSQL's built-in full-text search via Django's contrib.postgres module is the right starting point — it supports multi-field search with weighted ranking.

```
from django.contrib.postgres.search import SearchVector, SearchRank, SearchQuery
```

```
vector = SearchVector('title', weight='A') + SearchVector('content', weight='B')
```

```
query = SearchQuery(user_input)
```

```
Post.objects.annotate(rank=SearchRank(vector, query)).filter(rank__gte=0.1).order_by('-rank')
```

Add a GinIndex on the search vector column for speed. For larger scale or more advanced ranking, swap Postgres FTS for Elasticsearch or Meilisearch.

Q13. If your app needs to send emails to 10,000 users, how would you handle that in Django?

Offload it to a background task queue — Celery with Redis as the broker is the standard approach. The view just enqueues the task and returns immediately; Celery workers handle the actual sending in the background.

```
# tasks.py
@shared_task
def send_bulk_emails(user_ids):
    for uid in user_ids:
        send_email(uid)

# views.py
send_bulk_emails.delay(user_ids) # non-blocking
```

This keeps the request fast for the user and lets the workers process the queue at their own pace. You can also add retries and rate limiting at the Celery level.

Q14. How would you expose a Django API to be consumed by a plain JavaScript frontend?

Use Django REST Framework — create a serializer, a view, and map it to a URL. The view returns JSON.

```
# views.py
@api_view(['GET'])
def posts(request):
    data = Post.objects.values('id', 'title')
    return Response(data)
```

On the JS side, just fetch('/api/posts/'). If the frontend is on a different origin, configure django-cors-headers to allow cross-origin requests. Use JSON Web Tokens (JWT) for auth if needed.

Q15. How would you update parts of a Django webpage dynamically using JavaScript without a full refresh?

Fetch data from a Django endpoint and update the DOM directly using JavaScript.

```
fetch('/api/data/')
  .then(r => r.json())
  .then(data => {
    document.getElementById('result').innerText = data.value;
  });
```

On the Django side, the view returns a JsonResponse. No page reload — just the specific element updates. For more complex scenarios, this same pattern scales to WebSockets or polling.

Q16. Have you worked with vector databases like ChromaDB or Qdrant? What are they and how do they differ from SQL?

Yes — I've used both ChromaDB and Qdrant while building RAG pipelines in my Sports Analytics AI Assistant project.

Vector databases store data as high-dimensional embeddings and are designed for semantic similarity search — 'find me documents closest in meaning to this query.' SQL databases store structured rows and handle exact, rule-based queries.

SQL: 'Give me all users where age > 25'

Vector DB: 'Give me documents semantically similar to this sentence'

They're complementary, not competitive. In AI applications — especially RAG — you typically use both: a vector DB for semantic retrieval and a SQL DB for structured metadata filtering.

Q17. What is the difference between synchronous and asynchronous JavaScript?

Synchronous code runs line by line and blocks until each step finishes before moving to the next. Asynchronous code lets you kick off a task and move on — the result comes back later via a Promise or callback, without freezing the program.

```
// Synchronous — blocks everything until file is read
const data = fs.readFileSync('file.txt');
```

```
// Asynchronous — non-blocking
const data = await fs.promises.readFile('file.txt');
```

Since JavaScript is single-threaded, async is essential for I/O operations (API calls, file reads, DB queries) — otherwise the entire thread would freeze waiting.

Q18. How would you pass data from a Django view to a template and then access it in JavaScript?

Pass data through the view's context, then use the `json_script` template filter to safely serialize it into the page.

```
# views.py
return render(request, 'page.html', {'posts': list(Post.objects.values())})

<!-- template -->
{{ posts|json_script:"posts-data" }}
<script>
  const posts = JSON.parse(document.getElementById('posts-data').textContent);
</script>
```

`json_script` safely escapes the data and prevents XSS. Avoid dropping raw template variables directly inside `<script>` tags.

Q19. How would you select all elements with a specific class and attach a click event listener to each?

```
document.querySelectorAll('.my-class').forEach(el => {  
  el.addEventListener('click', () => {  
    console.log('Clicked:', el.textContent);  
  });  
});
```

querySelectorAll returns a NodeList of all matching elements. forEach iterates over each and attaches an independent event listener.

Q20. Name one example of good UI and one example of bad UI. Mention the websites.

Good UI — Linear (linear.app):

Clean, fast, and keyboard-first. Every action is exactly where you'd expect it. Nothing clutters the screen asking for attention. It respects the user's workflow instead of interrupting it.

Bad UI — IRCTC (irctc.co.in):

Too much happening on a single screen, confusing navigation, and the booking flow has several unnecessary steps. It's easy to lose track of where you are in the process. The interface doesn't guide the user — it makes them fight through it.

Submitted by: Dhairya Rathore

B.Tech — ECE, IIIT Nagpur

rathoredhairya2004@gmail.com | +91 8233798976

Submission Date: 30 May 2026